

K-Concatenation Maximum Sum

Absolutely. Let's understand **LeetCode 1191 — K-Concatenation Maximum Sum** from the intuition first, rather than jumping directly to the formula.

1. What is the problem asking?

You are given:

```
arr = [1, 2]
k = 3
```

We repeat the array k times:

```
[1, 2] + [1, 2] + [1, 2]
= [1, 2, 1, 2, 1, 2]
```

Now we need to find the **maximum sum of any contiguous subarray**.

For this example:

```
[1, 2, 1, 2, 1, 2]
```

The whole array gives:

```
1 + 2 + 1 + 2 + 1 + 2 = 9
```

So answer = 9.

Also, because an empty subarray is allowed:

```
maximum sum can never be less than 0
```

For example:

```
arr = [-1, -2]
```

Every non-empty subarray has a negative sum, so we choose the empty subarray:

```
answer = 0
```

2. The obvious/brute-force idea

Suppose:

```
arr = [1, 2]
k = 3
```

We could actually construct:

```
[1, 2, 1, 2, 1, 2]
```

Then run **Kadane's Algorithm** on it.

Kadane's Algorithm finds the maximum subarray sum in:

```
O(n)
```

But the concatenated array has:

```
n * k
```

elements.

So this approach takes:

```
O(n * k)
```

Given:

```
n <= 10^5
k <= 10^5
```

we could have:

```
n * k = 10^10
```

which is far too large.

So we need to avoid actually constructing the repeated array.

3. The key observation

The important question is:

What can the maximum subarray look like when we repeat the array many times?

Consider:

```
arr = [a, b, c, d]
```

and:

```
arr + arr + arr + ...
```

A maximum subarray can have three important forms.

Case 1: It is completely inside one copy

For example:

```
[a, b, c, d]
      ↑
      [c,d]
```

Then we simply need the normal maximum subarray sum of `arr`.

Case 2: It crosses from one copy into the next

For example:

```
[a, b, c, d] [a, b, c, d]
      ↑       ↑
    suffix  prefix
```

Suppose the best subarray is:

```
[c, d, a, b]
```

This consists of:

```
best suffix of arr
+
best prefix of arr
```

So we need:

```
maxSuffix + maxPrefix
```

Case 3: It goes through many complete copies

For example:

```
[a,b,c,d] [a,b,c,d] [a,b,c,d] [a,b,c,d]
      ↑           ↑           ↑           ↑
      └──────────┘
```

The middle part may contain entire copies of the array.

If:

```
sum(arr) > 0
```

then taking an entire copy is beneficial.

So if we have `k` copies, we can potentially take:

```
bestSuffix
+ totalSum * (k - 2)
+ bestPrefix
```

Why `k - 2`?

Because:

```
first copy  → suffix
middle copies → complete arrays
last copy   → prefix
```

For $k = 5$:

```

copy 1      copy 2      copy 3      copy 4      copy 5
suffix      complete      complete      complete      prefix
              ↑
            3 complete copies

```

So:

$$k - 2 = 5 - 2 = 3$$

4. Why do we only care about the prefix and suffix?

Imagine:

```
arr = [1, 2, 3, 4]
```

Suppose a subarray crosses the boundary:

[1, 2, 3, 4] [1, 2, 3, 4]
 ↑ ↑

Anything from the first copy that is taken must be a **suffix**.

Anything from the second copy that is taken must be a **prefix**.

So a boundary-crossing subarray always looks like:

suffix + prefix

If it goes across multiple copies:

suffix + complete copies + prefix

That's the central idea of the problem.

5. Three things we need to calculate

We need:

① Maximum subarray sum

Use Kadane's Algorithm.

maxSubarray

② Maximum prefix sum

Example:

```
arr = [1, 2, -5, 4]
```

Possible prefixes:

$$\begin{aligned} [1] &= 1 \\ [1, 2] &= 3 \\ [1, 2, -5] &= -2 \\ [1, 2, -5, 4] &= 2 \end{aligned}$$

So:

```
maxPrefix = 3
```

③ Maximum suffix sum

Possible suffixes:

$$\begin{aligned} [4] &= 4 \\ [-5, 4] &= -1 \\ [2, -5, 4] &= 1 \end{aligned}$$

```
[1,2,-5,4] = 2
```

So:

```
maxSuffix = 4
```

6. What about the total sum?

Suppose:

```
arr = [1, 2]
```

Then:

```
totalSum = 3
```

If we have:

```
k = 5
```

the repeated array is:

```
[1,2] [1,2] [1,2] [1,2] [1,2]
```

Since every complete copy adds 3, taking more copies keeps increasing our answer.

So we can use:

```
totalSum * (k - 2)
```

But what if:

```
totalSum < 0
```

Then taking additional complete copies makes the answer **smaller**.

Therefore, we should take:

```
C++
max(0LL, totalSum)
```

So the formula becomes:

```
maxSuffix
+
maxPrefix
+
max(0, totalSum) * (k - 2)
```

for $k \geq 2$.

7. But there is one more case

Suppose:

```
arr = [1, -2, 1]
```

and:

```
k = 5
```

The total sum is:

```
1 - 2 + 1 = 0
```

The best answer is:

```
2
```

from:

```
[1, -2, 1, 1, -2, 1, ...]
      ↑
      1 + 1
```

So we cannot simply use:

```
prefix + suffix
```

because there could be a maximum subarray completely inside one or two copies.

Therefore, for $k \geq 2$, we need to consider:

1. Maximum subarray inside two copies
2. Prefix + suffix + complete middle copies

This is why we calculate Kadane on:

```
arr + arr
```

but we **do not actually construct it**.

We simply iterate through the array twice.

8. The final formula

If $k == 1$

Just run normal Kadane:

```
answer = maxSubarray(arr)
```

If $k \geq 2$

Calculate:

```
bestTwo = maximum subarray sum in arr + arr
```

Then calculate:

```
candidate =
    maxPrefix
    + maxSuffix
    + max(0, totalSum) * (k - 2)
```

Finally:

```
answer = max(bestTwo, candidate)
```

And because the empty subarray is allowed:

```
answer >= 0
```

So we initialize Kadane appropriately.

9. C++ Code

```
C++
class Solution {
public:
    static const long long MOD = 1000000007LL;

    long long kadane(vector<int>& arr, int times) {
        long long current = 0;
        long long best = 0;

        int n = arr.size();

        for (int i = 0; i < n * times; i++) {
            current = max(0LL, current + arr[i % n]);
            best = max(best, current);
        }

        return best;
    }

    int kConcatenationMaxSum(vector<int>& arr, int k) {
        int n = arr.size();

        // Case 1: Only one copy
        if (k == 1) {
            return kadane(arr, 1) % MOD;
        }
    }
}
```

```
// Total sum of one copy
long long totalSum = 0;

for (int x : arr) {
    totalSum += x;
}

// Maximum prefix sum
long long prefix = 0;
long long maxPrefix = 0;

for (int x : arr) {
    prefix += x;
    maxPrefix = max(maxPrefix, prefix);
}

// Maximum suffix sum
long long suffix = 0;
long long maxSuffix = 0;

for (int i = n - 1; i >= 0; i--) {
    suffix += arr[i];
    maxSuffix = max(maxSuffix, suffix);
}

// Maximum subarray sum inside arr + arr
long long bestTwo = kadane(arr, 2);

// If totalSum is positive, middle copies help us.
long long middle = max(0LL, totalSum) * (k - 2);

long long answer = max(
    bestTwo,
    maxPrefix + maxSuffix + middle
);

return answer % MOD;
};
```

10. Let's understand the code piece by piece

kadane()

```
C++
long long current = 0;
long long best = 0;
```

current = maximum subarray sum ending at the current position.

best = maximum subarray sum found so far.

We start at 0 because the problem allows an empty subarray.

Then:

```
C++
current = max(0LL, current + arr[i % n]);
```

The interesting part is:

```
C++
arr[i % n]
```

This allows us to pretend that the array is repeated.

For:

```
arr = [1,2]
```

indices become:

i	i % n	element
0	0	1
1	1	2
2	0	1
3	1	2

So:

```
arr[i % n]
```

gives:

```
1 2 1 2
```

without actually creating another array.

11. Maximum prefix

```
C++
long long prefix = 0;
long long maxPrefix = 0;

for (int x : arr) {
    prefix += x;
    maxPrefix = max(maxPrefix, prefix);
}
```

Suppose:

```
arr = [1, -2, 3]
```

We get:

```
prefix:
1
1 + (-2) = -1
-1 + 3 = 2
```

Therefore:

```
maxPrefix = 2
```

12. Maximum suffix

We simply go from right to left:

```
C++
long long suffix = 0;
long long maxSuffix = 0;

for (int i = n - 1; i >= 0; i--) {
    suffix += arr[i];
    maxSuffix = max(maxSuffix, suffix);
}
```

For:

```
[1, -2, 3]
```

we get:

```
3
3 + (-2) = 1
1 + 1 = 2
```

Therefore:

```
maxSuffix = 3
```

13. Why `max(0LL, totalSum)`?

Consider:

```
arr = [1, -2]
```

Then:

```
totalSum = -1
```

Suppose:

```
k = 5
```

Taking complete copies gives:

```
-1
-1 + -1
-1 + -1 + -1
...
```

That's clearly bad.

So:

```
C++
max(0LL, totalSum)
```

turns:

```
-1 → 0
```

meaning:

"Don't take the middle complete copies if they decrease our sum."

14. Complete dry run — Example 1

Given:

```
arr = [1, 2]
k = 3
```

The actual repeated array would be:

```
[1, 2, 1, 2, 1, 2]
```

But our algorithm doesn't build it.

Step 1: Total sum

```
totalSum = 1 + 2
          = 3
```

So:

```
totalSum = 3
```

Step 2: Maximum prefix

Prefixes:

```
[1]    → 1
[1,2]  → 3
```

Therefore:

```
maxPrefix = 3
```

Step 3: Maximum suffix

Suffixes:

```
[2]    → 2
[1,2]  → 3
```

Therefore:

```
maxSuffix = 3
```

Step 4: Maximum subarray in two copies

Two copies:

```
[1, 2, 1, 2]
```

Kadane finds:

```
1 + 2 + 1 + 2 = 6
```

Therefore:

```
bestTwo = 6
```

Step 5: Middle copies

We have:

```
k = 3
```

So:

```
k - 2 = 1
```

And:

```
totalSum = 3
```

Therefore:

```
middle = 3 * 1
        = 3
```

Step 6: Boundary-crossing candidate

```
maxPrefix + maxSuffix + middle
```

=>

```
3 + 3 + 3
```

=>

```
9
```

So:

```
candidate = 9
```

Step 7: Final answer

Compare:

```
bestTwo = 6
candidate = 9
```

Therefore:

```
answer = max(6, 9)
        = 9
```

Final:

```
9
```

Which corresponds to taking the entire repeated array:

```
[1, 2] [1, 2] [1, 2]
```

Sum:

```
3 + 3 + 3 = 9
```

15. Quick dry run — Example 2

```
arr = [1, -2, 1]
k = 5
```

Total:

```
1 - 2 + 1 = 0
```

Maximum prefix:

```
1
1 + (-2) = -1
1 + (-2) + 1 = 0
maxPrefix = 1
```

Maximum suffix:

```
1
-2 + 1 = -1
1 - 2 + 1 = 0
maxSuffix = 1
```

Since:

```
totalSum = 0
```

middle copies don't help:

```
middle = 0
```

So:

```
candidate = 1 + 1 + 0
           = 2
```

And the maximum subarray within two copies is also at least 2.

Therefore:

```
answer = 2
```

16. Example 3

```
arr = [-1, -2]
k = 7
```

Everything is negative.

Therefore the best non-empty subarray is negative.

But the problem says an empty subarray is allowed.

So:

```
answer = 0
```

Our Kadane starts with:

```
C++
current = 0;
best = 0;
```

and therefore correctly returns:

```
0
```

17. The main intuition to remember

This problem looks difficult because k can be huge.

The trick is:

Don't think about all k copies individually. Think about what a maximum subarray can look like around the boundaries.

A maximum subarray can be:

```
Inside one/two copies
```

or:

```
suffix + complete copies + prefix
```

So we only need:

```
Maximum subarray in 2 copies
+
Maximum prefix
+
Maximum suffix
+
Total sum
```

The complete copies are handled mathematically:

```
totalSum * (k - 2)
```

instead of physically creating them.

18. Complexity

Let:

```
n = arr.size()
```

We make a constant number of passes over the array.

Time

```
O(n)
```

Even if:

```
k = 100000
```

we **don't** loop **k** times except conceptually through the formula.

Space

```
O(1)
```

We don't construct the **n * k** array.

★ Final formula to remember

For **k = 1**:

```
answer = Kadane(arr)
```

For **k >= 2**:

```
answer = max(
    maximumSubarray(arr + arr),
    maximumPrefix(arr)
    + maximumSuffix(arr)
    + max(0, totalSum) * (k - 2)
)
```

And finally:

```
answer %= 1e9 + 7
```

The most important idea: **k** may be enormous, but the structure of the best subarray is simple enough that we can represent all those repeated copies using the **total sum**, rather than actually constructing them.

Yes — this is the **most confusing part** of this problem. Let's forget the formula for a moment and understand **exactly when prefix and suffix are used**.

1. First, imagine two copies

Suppose:

```
arr = [1, 2, 3, 4]
```

After repeating:

```
[1, 2, 3, 4] [1, 2, 3, 4]
```

There is a **boundary** between the two copies:

```
[1, 2, 3, 4] | [1, 2, 3, 4]
           ↑
        boundary
```

Now imagine our maximum subarray crosses this boundary.

For example:

```
[1, 2, 3, 4] | [1, 2, 3, 4]
           ↑       ↑
        start   end
           [3, 4, 1, 2]
```

Look carefully.

From the **first copy**, we took:

```
[3, 4]
```

That is a **suffix** of the first copy.

From the **second copy**, we took:

```
[1, 2]
```

That is a **prefix** of the second copy.

Therefore:

```
suffix + prefix
```

2. Why can't it be prefix + prefix?

Because a subarray must be **continuous**.

Suppose:

```
[1, 2, 3, 4] | [1, 2, 3, 4]
```

If we start at 1 in the first copy and cross the boundary, we cannot suddenly stop at 2 before reaching the boundary.

We would have to take:

```
[1, 2, 3, 4] | ...
```

So if we enter the **next copy**, the part from the previous copy must reach its **end**.

That's why it is a suffix.

3. Here's the important condition

We use prefix + suffix ONLY when the maximum subarray crosses the boundary between copies.

That's the condition.

Think:

```
Copy 1          Copy 2
[1 2 3 4] | [1 2 3 4]
           ↑
        boundary
```

If our subarray stays entirely inside one copy:

```
[1 2 [3 4]] | [1 2 3 4]
```

Then we don't need prefix/suffix.

We just use normal Kadane.

But if it crosses:

```
[1 2 [3 4] | [1 2] 3 4]
           ↑       ↑
        suffix  prefix
```

then:

```
answer = suffix + prefix
```

4. Let's use a better example

Take:

```
arr = [-5, 4, 3, -2]
```

Repeated twice:

```
[-5, 4, 3, -2] | [-5, 4, 3, -2]
```

Suppose we choose:

```
[4, 3, -2, -5, 4, 3]
```

Let's separate it:

```
[-5, [4, 3, -2]] | [[-5, 4, 3], -2]
      ↑           ↑
    suffix      prefix
```

First part:

```
[4, 3, -2]
```

is a suffix of the first array.

Second part:

```
[-5, 4, 3]
```

is a prefix of the second array.

Therefore:

```
sum = suffix + prefix
```

5. Now here's where maxPrefix and maxSuffix come from

Suppose:

```
arr = [-5, 4, 3, -2]
```

Prefixes:

```
[-5]           = -5
[-5,4]         = -1
[-5,4,3]       = 2
[-5,4,3,-2]    = 0
```

Best prefix:

```
maxPrefix = 2
```

Suffixes:

```
[-2]           = -2
[3,-2]         = 1
[4,3,-2]       = 5
[-5,4,3,-2]    = 0
```

Best suffix:

```
maxSuffix = 5
```

So if we cross the boundary, the best possible choice is:

```
best suffix + best prefix
```

```
5 + 2 = 7
```

And indeed:

```
[4, 3, -2] | [-5, 4, 3]
```

has:

```
5 + 2 = 7
```

6. But why do we take the maximum prefix/suffix?

This is another important point.

Suppose we know our subarray is going to cross the boundary.

We have a choice.

From the first copy, we could take:

```
[-2]
```

or:

```
[3, -2]
```

or:

```
[4, 3, -2]
```

or the entire array.

Which one should we choose?

Obviously, the one with the **largest sum**.

That's exactly:

```
maxSuffix
```

Similarly, after crossing into the second copy, we can take:

```
[-5]
[-5, 4]
[-5, 4, 3]
[-5, 4, 3, -2]
```

We choose whichever has the largest sum.

That's:

```
maxPrefix
```

Therefore:

```
best crossing subarray
=
maxSuffix + maxPrefix
```

7. Now bring k copies into the picture

Suppose:

```
arr = [1, 2]
k = 4
```

We have:

```
[1, 2] | [1, 2] | [1, 2] | [1, 2]
```

A subarray might look like:

```
[1, 2] | [1, 2] | [1, 2] | [1, 2]
```

If it starts somewhere in the first copy and ends somewhere in the last copy, its structure is:

```
suffix
+
complete copy
+
complete copy
+
```

```
prefix
```

So:

```
suffix + complete copies + prefix
```

For $k = 4$, there are:

```
k - 2 = 2
```

complete middle copies.

Therefore:

```
maxSuffix
+ totalSum * 2
+ maxPrefix
```

8. The CONDITION for taking the middle copies

Now we get to the exact condition you're asking about.

Suppose:

```
arr = [1, 2]
```

Then:

```
sum(arr) = 3
```

Every complete copy gives us:

```
+3
```

So taking more copies is beneficial.

We want:

```
suffix + 3 + 3 + prefix
```

But suppose:

```
arr = [-5, 2]
```

Then:

```
sum(arr) = -3
```

A complete copy gives:

```
-3
```

Why would we take it?

We wouldn't.

So we use:

```
C++
max(0LL, totalSum)
```

Meaning:

```
if totalSum > 0:
    take the complete copies
else:
    don't take them
```

9. Think of it like crossing bridges

This mental model might help.

Imagine each copy is an island:

```
Island 1   Island 2   Island 3
[ array ] → [ array ] → [ array ]
```

If your subarray starts on Island 1 and ends on Island 3:

```
START                END
  ↓                  ↓
[ array ] → [ array ] → [ array ]
```

On the first island, you can only take the **ending part**:

```
suffix
```

On the last island, you can only take the **starting part**:

```
prefix
```

And the islands in the middle must be taken **completely**.

So:

```
first copy → suffix
middle    → complete arrays
last copy → prefix
```

That's the entire reason for prefix and suffix.

10. One very important distinction

Don't think:

"We always take prefix and suffix."

That's **not** true.

Think:

If the optimal subarray crosses from one copy into another, then its first-copy portion must be a suffix and its last-copy portion must be a prefix.

If the best subarray doesn't cross a boundary, normal Kadane handles it.

That's why our solution compares:

```
bestTwo
```

with:

```
maxSuffix + middle + maxPrefix
```

bestTwo

Handles cases where the answer is simply inside one/two copies.

prefix + middle + suffix

Handles cases where the answer stretches across multiple copies.

The picture to remember

```
Copy 1           Middle copies           Last copy
[ ... SUFFIX ] | [ COMPLETE ] [ COMPLETE ] | [ PREFIX ... ]
    ↑                ↑                ↑
  partial      full arrays      partial
```

So the condition is simply:

Does our chosen subarray cross a boundary between repeated copies?

- **No** → normal maximum subarray/Kadane.
- **Yes** → suffix + prefix.
- **Crosses multiple copies** → suffix + complete middle copies + prefix.

That's the key idea behind the entire problem.

Yes. Let's do a **full dry run** of:

```
arr = [1, -2, 1, 3]
k = 5
```

I'll focus especially on **why prefix and suffix are used** and where each value comes from.

1. First understand the actual repeated array

We have:

```
[1, -2, 1, 3]
```

and **k** = 5.

So if we actually expanded it:

```
[1, -2, 1, 3] |
[1, -2, 1, 3] |
[1, -2, 1, 3] |
[1, -2, 1, 3] |
[1, -2, 1, 3]
```

or:

```
1 -2 1 3 | 1 -2 1 3 | 1 -2 1 3 | 1 -2 1 3 | 1 -2 1 3
```

We do **not** actually create this array.

2. First calculate totalSum

Original array:

```
[1, -2, 1, 3]
```

Add everything:

```
1 + (-2) + 1 + 3
= 3
```

So:

```
totalSum = 3
```

This is important because:

```
totalSum > 0
```

Therefore, taking a complete copy of the array **increases** our sum by 3.

Since:

```
k = 5
```

we potentially have:

```
k - 2 = 3
```

middle complete copies.

3. Find maximum prefix

Now let's calculate **maxPrefix**.

A prefix must start from the **first element**.

Array:

```
[1, -2, 1, 3]
```

Possible prefixes:

Prefix 1

```
[1]
```

sum:

1

So:

$\text{maxPrefix} = 1$

Prefix 2

[1, -2]

sum:

$1 + (-2) = -1$

Compare:

$\text{max}(1, -1) = 1$

So:

$\text{maxPrefix} = 1$

Prefix 3

[1, -2, 1]

sum:

$1 - 2 + 1 = 0$

Still:

$\text{maxPrefix} = 1$

Prefix 4

[1, -2, 1, 3]

sum:

$1 - 2 + 1 + 3 = 3$

Now:

$\text{maxPrefix} = 3$

So finally:

$\text{maxPrefix} = 3$

The best prefix is:

[1, -2, 1, 3]

with sum 3.

4. Find maximum suffix

Now we go from **right to left**.

Remember:

A suffix must end at the **last element**.

Array:

[1, -2, 1, 3]

Suffix 1

```
[3]
```

sum:

```
3
```

So:

```
maxSuffix = 3
```

Suffix 2

```
[1, 3]
```

sum:

```
1 + 3 = 4
```

Now:

```
maxSuffix = 4
```

Suffix 3

```
[-2, 1, 3]
```

sum:

```
-2 + 1 + 3 = 2
```

Still:

```
maxSuffix = 4
```

Suffix 4

```
[1, -2, 1, 3]
```

sum:

```
1 - 2 + 1 + 3 = 3
```

Still:

```
maxSuffix = 4
```

Therefore:

```
maxSuffix = 4
```

The best suffix is:

```
[1, 3]
```

with sum:

```
4
```

5. Now let's understand WHY this suffix and prefix matter

This is the important part.

Imagine two copies:

```
Copy 1          Copy 2
[1 -2 1 3] | [1 -2 1 3]
```

Suppose we want a subarray that crosses the boundary.

The best suffix from Copy 1 is:

```
[1, 3]
```

with sum 4.

The best prefix from Copy 2 is:

```
[1, -2, 1, 3]
```

with sum 3.

Therefore one possible boundary-crossing subarray is:

```
[1, 3] | [1, -2, 1, 3]
```

Its sum is:

```
4 + 3 = 7
```

Notice how it is:

```
SUFFIX | PREFIX
[1,3]  | [1,-2,1,3]
```

That's exactly why we're calculating them.

6. But we have 5 copies

Our array is:

```
Copy 1 | Copy 2 | Copy 3 | Copy 4 | Copy 5
```

A subarray that starts somewhere in Copy 1 and ends somewhere in Copy 5 could look like:

```
SUFFIX | COMPLETE | COMPLETE | COMPLETE | PREFIX
```

For our array:

```
[1,3] | [1,-2,1,3] | [1,-2,1,3] | [1,-2,1,3] | [1,-2,1,3]
```

Let's add it.

Suffix:

```
4
```

Three middle complete copies:

```
3 + 3 + 3 = 9
```

Prefix:

```
3
```

Total:

```
4 + 9 + 3 = 16
```

So our candidate is:

```
candidate = 16
```

7. Why exactly are there 3 middle copies?

We have:

```
5 copies
```

The first copy contributes a suffix.

The last copy contributes a prefix.

That leaves:

```
5 - 2 = 3
```

copies in the middle.

Therefore:

```
middle = totalSum * (k - 2)
```

```
middle = 3 * 3
        = 9
```

8. Now let's verify with the actual expanded array

The actual array would be:

```
1 -2 1 3 | 1 -2 1 3 | 1 -2 1 3 | 1 -2 1 3 | 1 -2 1 3
```

Our proposed subarray is:

```
1 -2 [1 3 | 1 -2 1 3 | 1 -2 1 3 | 1 -2 1 3 | 1 -2 1 3]
```

Let's write it as one sequence:

```
[1, 3, 1, -2, 1, 3, 1, -2, 1, 3, 1, -2, 1, 3]
```

Now sum:

```
1 + 3
= 4

+ 1 - 2 + 1 + 3
= 7

+ 1 - 2 + 1 + 3
= 10

+ 1 - 2 + 1 + 3
= 13

+ 1 - 2 + 1 + 3
= 16
```

So yes:

```
answer = 16
```

9. But wait — what about bestTwo?

Our algorithm also calculates the maximum subarray sum in **two copies**.

Two copies are:

```
1 -2 1 3 | 1 -2 1 3
```

Let's run Kadane properly.

We'll track:

```
current = maximum subarray sum ending here
best = maximum found so far
```

Start:

```
current = 0
best = 0
```

Element 1

```
current = max(0, 0 + 1)
        = 1
```

```
best = max(0, 1)
      = 1
```

Element -2

```
current = max(0, 1 - 2)
        = 0
```

```
best = max(1, 0)
      = 1
```

Element 1

```
current = max(0, 0 + 1)
          = 1
```

```
best = 1
```

Element 3

```
current = max(0, 1 + 3)
          = 4
```

```
best = 4
```

Element 1 from Copy 2

```
current = max(0, 4 + 1)
          = 5
```

```
best = 5
```

Our subarray is now:

```
[1, 3, 1]
```

with sum:

```
5
```

Element -2

```
current = max(0, 5 - 2)
          = 3
```

```
best = 5
```

Element 1

```
current = max(0, 3 + 1)
          = 4
```

Element 3

```
current = max(0, 4 + 3)
          = 7
```

So:

```
bestTwo = 7
```

Thus:

```
bestTwo = 7
```

10. Now compare the two possibilities

We have:

```
bestTwo = 7
```

and:

```
suffix + middle + prefix
=
4 + 9 + 3
=
16
```

So:

```
answer = max(7, 16)
         = 16
```

11. Why doesn't `bestTwo = 7` already solve everything?

Because `bestTwo` only considers **at most two copies**.

But here:

```
totalSum = 3 > 0
```

So every extra complete copy gives us another:

```
+3
```

We have 5 copies, so using more copies can make the answer much bigger.

That's why:

```
suffix + middle copies + prefix
```

is necessary.

12. The most important visualization

For this exact example:

```
arr = [1,-2,1,3]
k = 5
```

Think of the optimal subarray as:

Copy 1	Copy 2	Copy 3	Copy 4	Copy 5
[1, -2, 1,3]	[1,-2,1,3]	[1,-2,1,3]	[1,-2,1,3]	[1,-2,1,3]
↑ suffix	↑ complete	↑ complete	↑ complete	↑ prefix

More accurately, the selected part is:

[1,3]	[1,-2,1,3]	[1,-2,1,3]	[1,-2,1,3]	[1,-2,1,3]
↑ suffix	↑ complete	↑ complete	↑ complete	↑ prefix

Their sums are:

```
suffix    = 4
complete  = 3
complete  = 3
complete  = 3
prefix    = 3
```

Therefore:

```
4 + 3 + 3 + 3 + 3
= 16
```

13. The key condition, in THIS example

We use:

```
maxSuffix + totalSum*(k-2) + maxPrefix
```

because **all three conditions are true**:

Condition 1

We are considering a subarray that crosses between copies.

Therefore:

```
suffix + prefix
```

Condition 2

There are multiple copies between the starting and ending copies.

Therefore:

```
middle complete copies
```

Condition 3

`totalSum > 0`.

Here:

```
totalSum = 3
```

Therefore those middle copies are worth taking.

So:

```
answer
= maxSuffix
+ totalSum*(k-2)
+ maxPrefix
```

```
= 4 + 3*3 + 3
= 16
```

One subtle point

Notice that **prefix/suffix** are not "extra elements we randomly decide to take."

They describe the **shape of a subarray that crosses copy boundaries**:

```
first copy  → suffix
middle      → whole copies
last copy   → prefix
```

For your example, the optimal shape is literally:

```
[1,3] | [1,-2,1,3] | [1,-2,1,3] | [1,-2,1,3] | [1,-2,1,3]
```

which gives 16.

Yes. Let's take an example where **total sum is negative**, because this makes the `max(0, totalSum)` part very clear.

We'll use:

```
arr = [4, -10, 3]
k = 5
```

1. First, repeat the array

If we actually expanded it:

```
[4, -10, 3] | [4, -10, 3] | [4, -10, 3] | [4, -10, 3] | [4, -10, 3]
```

2. Calculate total sum

```
4 + (-10) + 3
= -3
```

So:

```
totalSum = -3
```

This is the important situation.

Every time we take a **complete copy** of the array, we add:

```
-3
```

to our sum.

So taking more complete copies is actually **bad**.

For example:

```
1 complete copy → -3
2 complete copies → -6
3 complete copies → -9
```

Therefore, we don't want the middle complete copies.

3. Why do we use `max(0, totalSum)`?

Our formula contains:

```
C++
max(0LL, totalSum)
```

Since:

```
totalSum = -3
```

we get:

```
max(0, -3)
= 0
```

Therefore:

```
middle = 0 * (k - 2)
= 0
```

This means:

Don't take any complete middle copies.

This is the key point when `totalSum < 0`.

4. Calculate maximum prefix

Array:

```
[4, -10, 3]
```

Prefixes:

```
[4]           = 4
[4, -10]      = -6
[4, -10, 3]   = -3
```

Therefore:

```
maxPrefix = 4
```

5. Calculate maximum suffix

Suffixes:

```
[3]           = 3
[-10, 3]      = -7
[4, -10, 3]   = -3
```

Therefore:

```
maxSuffix = 3
```

6. Now what happens with the repeated array?

We have:

```
[4, -10, 3] | [4, -10, 3] | [4, -10, 3] | [4, -10, 3] | [4, -10, 3]
```

Since total sum is negative, we **don't want to take complete copies in the middle**.

So the boundary-crossing candidate is:

```
maxSuffix + maxPrefix
```

because:

```
middle = 0
```

Therefore:

```
3 + 4
= 7
```

So:

```
candidate = 7
```

7. What does this actually mean?

The suffix is:

```
[3]
```

from one copy.

The prefix is:

```
[4]
```

from the next copy.

So across the boundary we get:

```
[4, -10, 3] | [4, -10, 3]
      ↑
    boundary
      [3] | [4]
```

The selected subarray is:

```
[3, 4]
```

Sum:

```
3 + 4 = 7
```

Notice something important:

We **didn't take the middle copies** because every complete copy has sum -3.

8. But what about bestTwo?

We also calculate the maximum subarray in two copies:

```
[4, -10, 3] | [4, -10, 3]
```

Let's run Kadane.

Start

```
current = 0
best = 0
```

First 4

```
current = max(0, 0 + 4)
          = 4
best = 4
```

-10

```
current = max(0, 4 - 10)
          = 0
best = 4
```

3

```
current = max(0, 0 + 3)
          = 3
best = 4
```

Second 4

```
current = max(0, 3 + 4)
          = 7
best = 7
```

The subarray is:

```
[3, 4]
```

crossing the boundary.

-10

```
current = max(0, 7 - 10)
          = 0
```

3

```
current = max(0, 0 + 3)
          = 3
```

Final:

```
bestTwo = 7
```

9. Now calculate the final answer

We have:

```
bestTwo = 7
```

and:

```
maxSuffix = 3
maxPrefix = 4
totalSum = -3
k = 5
```

Middle:

```
max(0, -3) * (5 - 2)
= 0 * 3
= 0
```

Therefore:

```
candidate
= maxSuffix + middle + maxPrefix
= 3 + 0 + 4
= 7
```

Finally:

```
answer = max(bestTwo, candidate)
        = max(7, 7)
        = 7
```

Final answer: 7

10. Why don't we use all 5 copies?

This is the **main idea**.

The repeated array is:

```
[4, -10, 3] | [4, -10, 3] | [4, -10, 3] | [4, -10, 3] | [4, -10, 3]
```

Every complete copy gives:

```
4 - 10 + 3 = -3
```

So if we take:

```
[4, -10, 3]
```

we lose 3.

If we take three middle copies:

```
-3 + -3 + -3 = -9
```

That's terrible.

Instead, we take:

```
[3] | [4]
```

giving:

```
7
```

So the optimal subarray only crosses **one boundary**.

11. Compare positive vs negative total sum

This is the easiest way to remember it.

Case A: totalSum > 0

Example:

```
arr = [1, 2]
sum = 3
```

Every complete copy gives:

```
+3
```

So:

```
Take the middle copies.
```

Formula:

```
maxSuffix + totalSum * (k-2) + maxPrefix
```

Case B: totalSum < 0

Example:

```
arr = [4, -10, 3]
sum = -3
```

Every complete copy gives:

```
-3
```

So:

```
Don't take the middle copies.
```

Formula effectively becomes:

```
maxSuffix + maxPrefix
```

because:

```
C++
max(0LL, totalSum)
```

becomes 0.

Case C: totalSum == 0

Example:

```
arr = [1,-2,1]
sum = 0
```

A complete copy neither helps nor hurts.

So:

```
max(0, totalSum) = 0
```

and again:

```
middle = 0
```

Remember this one rule

```
totalSum > 0 → complete copies HELP → take them
totalSum < 0 → complete copies HURT → don't take them
totalSum = 0 → complete copies don't matter → middle contributes 0
```

And that's exactly why the formula contains:

```
C++
max(0LL, totalSum) * (k - 2)
```

The `max(0, ...)` is basically saying:

"If a whole copy helps me, take it. If it hurts me, skip it."

ChatGPT can make mistakes. Check important info.